

Java RMI - Demo

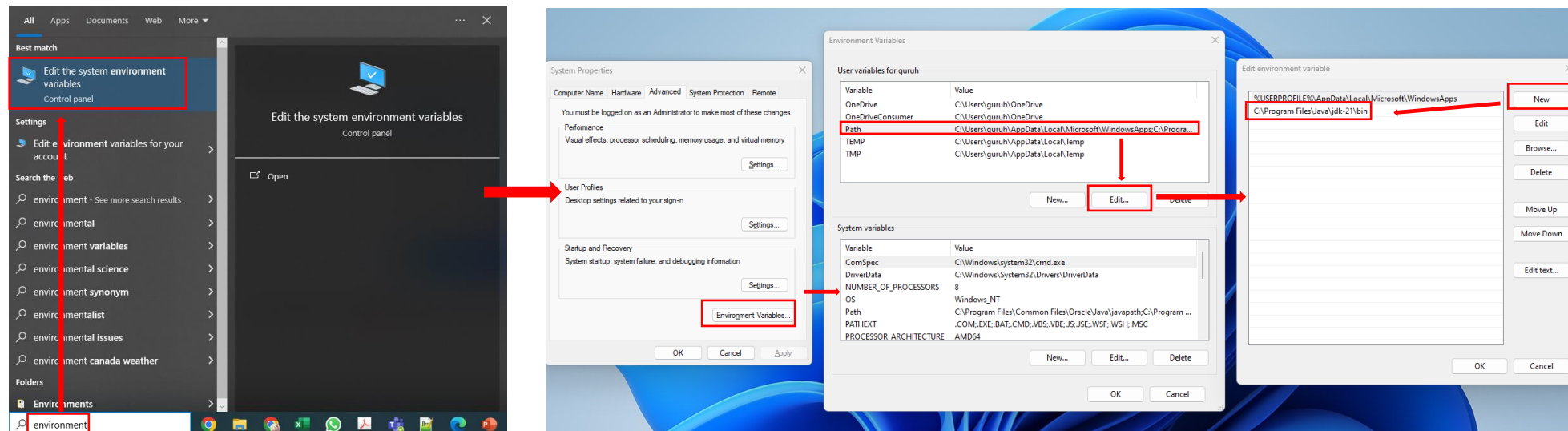
- Definisikan antarmuka remote
- Implementasikan server
- Implementasikan klien
- Compile file sumber
- Mulai registri Java RMI, server, dan klien

Preparation - Requirement

- Kebutuhan
 - **Java Development Kit (JDK).**
 - <https://www.oracle.com/java/technologies/downloads/>
 - Dalam sistem Operasi Microsoft Windows, JDK bisa terinstall dalam beberapa versi.
 - Unduh versi yang terbaru.
 - **Text editor** (Notepad, Notepad++, Vi, Nano, dll) / **IDE** (Visual Studio Code, NetBeans, Komodo Edit, Sublime Text, dll)

Preparation – step-by-step

- Install JDK yang sudah diunduh.
- **Pastikan PATH 'bin' Java/JDK telah didefinisikan/dimasukkan ke "Environment Variables" (Microsoft Windows)**
 - "PATH 'bin' Java" adalah alamat *folder* 'bin' yang terdapat dalam *folder* instalasi JAVA/JDK. Default ada di 'C:\Program Files\Java\jdk-21\bin'



- Buat folder baru bernama "RMI" di drive C:\

Implementasikan server

- Objek remote adalah instans dari sebuah kelas yang mengimplementasikan antarmuka remote.
- Sebuah antarmuka remote memperluas antarmuka *java.rmi.Remote* dan mendeklarasikan sejumlah metode remote.
- **Copy-Paste kode di samping, simpan sebagai file "Hello.java" dan letakkan di C:\RMI**

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
public interface Hello extends Remote {  
    String sayHello() throws RemoteException;  
}
```

Implementasikan server (Cont.)

- Kelas "server" dalam konteks ini adalah kelas yang memiliki metode utama yang membuat suatu instans implementasi objek remote, mengeksport objek remote, dan kemudian mengikat instans tersebut ke suatu nama dalam registri Java RMI.
- Metode utama dari server melakukan hal-hal berikut:
 - Membuat dan meng-ekspor objek remote

```
Server obj = new Server();  
Hello stub = (Hello) UnicastRemoteObject.exportObject(obj, 0);
```

- Daftarkan objek remote dengan registri Java RMI (registri *default* pada port **TCP 1099**)

```
Registry registry = LocateRegistry.getRegistry();  
registry.bind("Hello", stub);
```

Implementasikan server (Cont.)

- Kelas Server mengimplementasikan antarmuka jarak jauh Hello, menyediakan implementasi
- **Copy-Paste kode di samping, Simpan sebagai “Server.java” dan letakkan di C:\RMI.**

```
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class Server implements Hello {

    public Server() {}

    public String sayHello() {
        return "Hello, world!";
    }

    public static void main(String args[]) {

        try {
            Server obj = new Server();
            Hello stub = (Hello) UnicastRemoteObject.exportObject(obj, 0);

            // Bind the remote object's stub in the registry
            Registry registry = LocateRegistry.getRegistry();
            registry.bind("Hello", stub);

            System.err.println("Server ready");
        } catch (Exception e) {
            System.err.println("Server exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

Implementasikan klien

- Program klien memperoleh:
 - Sebuah *stub* untuk registri di host server,
 - Mencari *stub* objek remote berdasarkan nama di dalam registri,
 - Kemudian memanggil metode “sayHello” pada objek remote menggunakan *stub*.
- **Copy-Paste kode di samping ,
simpan sebagai file "Client.java"
dan letakkan di C:\RMI**

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Client {

    private Client() {}

    public static void main(String[] args) {

        String host = (args.length < 1) ? null : args[0];
        try {
            Registry registry = LocateRegistry.getRegistry(host);
            Hello stub = (Hello) registry.lookup("Hello");
            String response = stub.sayHello();
            System.out.println("response: " + response);
        } catch (Exception e) {
            System.err.println("Client exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

Compile file sumber

- Kita memiliki 3 *file* di bawah C:\RMI Hello.java
 - Sebuah antarmuka remote Server.java,
 - Implementasi objek remote yang mengimplementasikan antarmuka remote Client.java,
 - Klien sederhana yang memanggil metode antarmuka remote“

Untuk meng-compile, buka Command Prompt, dan ketikkan :

```
C:\RMI> javac -d dist Hello.java Server.java Client.java
```

Jalankan sistem

- *Runtime* sisi klien membuka koneksi ke server menggunakan informasi host dan port dalam *stub* objek remote, dan kemudian melakukan serialisasi data panggilan.
- *Runtime* sisi server menerima panggilan masuk, mengirimkan panggilan ke objek remote, dan melakukan serialisasi hasil (string *reply* "Hello, world!") kembali ke klien.
- *Runtime* sisi klien menerima, melakukan deserialisasi, dan mengembalikan hasil ke pemanggil.

Menjalankan Java RMI registry, server, and client

- Jalankan Java RMI registry

```
C:\RMI> set CLASSPATH=C:\RMI\dist
C:\RMI> start rmiregistry
```

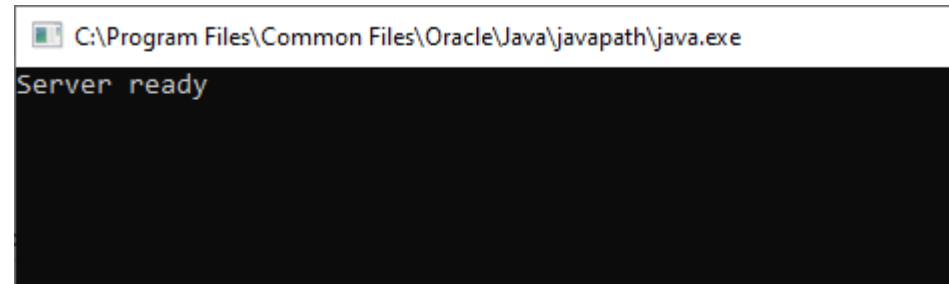
 - *RMI Registry* bertindak sebagai perantara antara server RMI dan klien.
 - ***RMI Registry* hanya bersifat sementara. Apabila *prompt* ditutup, maka perintah di atas harus diulang lagi.**
 - Server "mendaftarkan" layanannya di registri - oleh karena itu, RMI Registry dapat bertindak sebagai "direktori" untuk server-server/layanannya.
 - Klien tidak perlu mengetahui lokasi server-server individu, dan melakukan pencarian pada RMI Registry untuk layanan yang dibutuhkan.
 - Registri, sebagai direktori penamaan, mengembalikan penanganan yang sesuai kepada klien untuk memanggil metode-metode."
- Jalankan server

```
C:\RMI> start java -classpath dist -Djava.rmi.server.codebase=file:dist/ Server
```
- Jalankan klien

```
C:\RMI> java -classpath dist Client
```

Hasil

- Registri dan Server akan membuka *prompt* baru.
- Klien akan mencetak hasil panggilan



A screenshot of a Windows command prompt window. The title bar at the top reads "C:\Program Files\Common Files\Oracle\Java\javapath\java.exe". The main area of the window is black with white text that says "Server ready".

```
C:\RMI>set CLASSPATH=C:\RMI\dist
C:\RMI>start rmiregistry
C:\RMI>start java -classpath dist -Djava.rmi.server.codebase=file:dist/ Server
C:\RMI>java -classpath dist Client
response: Hello, world!
C:\RMI>
```